

RELATÓRIO ACADÊMICO

Título: Contagem de Números Primos e Análise de Desempenho Paralelo com OpenMP: Correção e Balanceamento de Carga **Autoria:** Thiago Jordão **Data:** 08/06/2026

RESUMO

Este relatório apresenta uma análise empírica e teórica sobre o comportamento de uma aplicação paralela desenvolvida em linguagem C utilizando a API OpenMP para a contagem de números primos. O estudo avalia um algoritmo limitado pela capacidade de processamento da CPU (*Compute-Bound*), contrastando o tempo de execução e a precisão dos resultados de versões puramente sequenciais contra versões paralelizadas. A avaliação de desempenho foi realizada variando o número de *threads* (de 1 a 12) sobre diferentes cargas de trabalho ($n = 500.000$ e $n = 1.000.000$). Os resultados demonstram na prática as graves consequências de ignorar dependências de dados em estruturas de repetição (gerando condições de corrida), bem como a limitação de escalabilidade imposta pelo desbalanceamento de carga quando não se utiliza o escalonamento adequado.

1. INTRODUÇÃO

A programação paralela surgiu como uma solução fundamental para contornar os limites físicos do aumento de frequência nos processadores, permitindo que múltiplas instruções sejam executadas simultaneamente. Ferramentas como o **OpenMP** facilitam a exploração do paralelismo em sistemas de memória compartilhada.

No entanto, o desenvolvimento de programas paralelos introduz desafios de correção e orquestração inexistentes no paradigma serial. O problema de encontrar e contar números primos através de tentativas de divisão sucessiva é um exemplo clássico de aplicação puramente *Compute-Bound*, onde o gargalo se encontra na disponibilidade de Unidades Lógicas e Aritméticas (ALU) do processador. O objetivo deste relatório é investigar como a paralelização de laços iterativos (*loop-level parallelism*) afeta a correção matemática da contagem e como a variação da carga computacional entre as iterações demanda estratégias avançadas de escalonamento dinâmico para garantir ganho de desempenho.

2. REFERENCIAL TEÓRICO

- **O Modelo OpenMP e a Diretiva `parallel for`:** O OpenMP é focado na paralelização de memória compartilhada através de diretivas de compilador. A diretiva `#pragma omp parallel for` é utilizada para instruir a equipe de *threads* a dividir as iterações de um laço `for` estruturado entre si. Por padrão, qualquer variável declarada antes do laço permanece com escopo compartilhado entre todas as *threads*.
- **Condições de Corrida (*Race Conditions*) e Dependências:** Se múltiplas *threads* tentarem atualizar simultaneamente a mesma variável global (ex: o acumulador de contagem de primos) sem o uso de proteção, ocorrerá uma condição de corrida que resultará na perda de incrementos e em resultados incorretos. O OpenMP resolve esse problema com a cláusula `reduction`, criando uma cópia local da variável de contagem para cada *thread* e, ao final, combinando-as numa seção crítica segura de forma transparente ao programador.

- **Escalaonamento e Balanceamento de Carga:** Por padrão, o OpenMP usa o escalonamento estático (`schedule(static)`), que divide o número total de iterações em grandes blocos iguais entre as *threads*. No entanto, testar se números altíssimos são primos exige ordens de grandeza mais tempo de CPU do que testar números baixos. Com um particionamento estático, *threads* responsáveis pelos primeiros números finalizam muito cedo e ficam ociosas. A solução teórica é a aplicação da cláusula `schedule(dynamic)`, onde as iterações são atribuídas às *threads* durante a execução ("sob demanda").

3. METODOLOGIA E CÓDIGO FONTE

Para validar a teoria, foi construído um programa em C para contar o número de primos entre 2 e um inteiro máximo n .

1. **Versão Sequencial:** Utilizou uma função de teste de divisão simples iterando os números de modo escalar.
2. **Versão Paralela:** A versão base iterou o espaço de busca utilizando a diretiva de divisão de laços do OpenMP (`#pragma omp parallel for`), construída deliberadamente sem proteção de acesso simultâneo para viabilizar a observação de condições de corrida.
3. **Avaliação:** Os programas foram testados variando n entre 500.000 e 1.000.000 e utilizando conjuntos de 1 a 12 *threads*. O tempo das seções foi aferido estritamente pelo uso da função `omp_get_wtime()`, garantindo a medição precisa do tempo de relógio global (*wall clock time*) e não do tempo de CPU.

O código fonte integral utilizado no experimento é apresentado a seguir:

```
#include <omp.h>
#include <stdio.h>
#include <stdlib.h>

static int is_prime(long n) {
    if (n < 2) {
        return 0;
    }

    for (long i = 2; i * i <= n; i++) {
        if (n % i == 0) {
            return 0;
        }
    }

    return 1;
}

static long count_primes_sequential(long n) {
    long count = 0;

    for (long i = 2; i <= n; i++) {
        if (is_prime(i)) {
            count++;
        }
    }
}
```

```
}

return count;
}

static long count_primes_parallel(long n) {
    long count = 0;

    #pragma omp parallel for
    for (long i = 2; i <= n; i++) {
        if (is_prime(i)) {
            count++;
        }
    }

    return count;
}

int main(int argc, char **argv) {
    long n = 500000;
    int num_threads = omp_get_max_threads();

    if (argc > 1) {
        n = strtol(argv, NULL, 10);
    }
    if (argc > 2) {
        num_threads = strtol(argv, NULL, 10);
    }

    if (n < 2 || num_threads < 1) {
        fprintf(stderr, "Uso: %s [n>=2] [numero_de_threads>=1]\n", argv);
        return 1;
    }

    omp_set_num_threads(num_threads);

    double start_seq = omp_get_wtime();
    long seq_count = count_primes_sequential(n);
    double seq_time = omp_get_wtime() - start_seq;

    double start_par = omp_get_wtime();
    long par_count = count_primes_parallel(n);
    double par_time = omp_get_wtime() - start_par;

    printf("Contagem de primos de 2 até %ld\n", n);
    printf("SEQUENCIAL -> primos: %ld | tempo(s): %.6f\n", seq_count, seq_time);
    printf("PARALELA -> primos: %ld | tempo(s): %.6f | threads: %d\n",
        par_count, par_time, omp_get_max_threads());

    if (seq_count == par_count) {
        printf("Resultado: OK (valores iguais)\n");
    } else {
        printf("Resultado: ERRO (valores diferentes)\n");
    }
}
```

```
if (par_time > 0.0) {  
    printf("Speedup (seq/par): %.2fx\n", seq_time / par_time);  
}  
  
return 0;  
}
```

4. RESULTADOS E DISCUSSÃO

4.1. Avaliação do Desempenho Isolado e o Overhead ($n = 500.000$) Os primeiros testes validaram o custo da paralelização usando a carga de trabalho de meio milhão de elementos.

Configuração	Tempo seq. (s)	Tempo par. (s)	Speedup
OMP_NUM_THREADS=1	0.056639	0.074840	0.76x
OMP_NUM_THREADS=8	0.056430	0.037893	1.49x

Discussão Teórica: Observa-se que a versão paralela rodando com apenas 1 *thread* foi visivelmente mais lenta que a versão serial direta (*speedup* de 0.76x). Isso comprova a presença de *overhead* estrutural: instanciar a equipe de *threads*, distribuir iterações e gerenciar as diretivas do OpenMP possui um custo que penaliza o desempenho quando não há real divisão de trabalho pelos múltiplos núcleos.

4.2. Avaliação das Dependências de Dados e Condição de Corrida ($n = 1.000.000$) A segunda bateria de testes expôs os riscos de manipular variáveis não-protegidas.

Threads	Tempo seq. (s)	Tempo par. (s)	Primos Enc.	Validação	Speedup
1	0.245000	0.212000	78498	OK	1.16x
2	0.247000	0.158000	78253	ERRO	1.56x
4	0.264000	0.101000	77366	ERRO	2.61x
6	0.235000	0.100000	77406	ERRO	2.35x
8	0.335000	0.100000	77391	ERRO	3.35x
12	0.240000	0.095000	77289	ERRO	2.53x

Discussão Teórica: Os dados acima refletem nitidamente uma falha grave de *Thread-Safety* no código. A partir do momento em que duas ou mais *threads* foram alocadas, a contagem de primos resultante foi sempre abaixo do valor real (78.498) e retornou totais diferentes a cada execução. A instrução `count++` da linha 37 não é atômica; se múltiplas *threads* tentarem acessar o registrador da variável no mesmo ciclo do relógio as gravações geram sobreposições, perdendo incrementos irremediavelmente. Para tratar este conflito, a diretiva deve ser complementada com a cláusula `reduction(+:count)`, que inicializa acumuladores privados e os consolida com segurança ao final.

4.3. O Paradoxo da Eficiência Computacional e Escalonamento Embora os tempos tenham melhorado com 8 *threads* (queda de cerca de 0.335s para 0.100s), o *speedup* geral ficou restrito em $\approx 3.35x$. Como o

problema é altamente *Compute-Bound*, um ganho tão distante do escalonamento linear puro indica desbalanceamento da carga. As *threads* responsáveis pelos números menores encerraram seu bloco subitamente e ficaram ociosas, sendo penalizadas pela carga contínua das *threads* que analisavam fatias de dados maiores e computacionalmente muito mais exigentes. Isto evidencia a necessidade de substituir a distribuição em bloco (`schedule(static)`) padrão pela distribuição sob demanda `schedule(dynamic)`.

5. CONCLUSÃO

A análise conduzida demonstrou que a inserção de paralelismo não apenas tem impactos no desempenho, mas pode comprometer fundamentalmente a integridade dos dados na ausência de perfilamento e sincronização corretos. O experimento comprova que:

1. Condições de corrida são implacáveis em blocos de iterações (`parallel for`) em que as atualizações cruzadas não são protegidas. A cláusula `reduction` é um requisito arquitetural obrigatório para operações de somatório ou contagem multithread com OpenMP.
2. Aplicações dependentes de processamento bruto (*Compute-Bound*) perdem escalabilidade quando a carga de trabalho de cada iteração não é uniforme. Sem orquestração explícita via `schedule(dynamic)`, recursos paralelos valiosos sofrem desperdício operando em estado ocioso e estagnam a eficiência paralela geral do sistema.

Em suma, a transição do código serial para o alto desempenho assenta-se inseparavelmente sobre a mitigação das dependências entre iterações e o balanceamento cirúrgico entre os núcleos.

REFERÊNCIAS

- PACHECO, Peter S. **An Introduction to Parallel Programming**. Morgan Kaufmann Publishers, 2011.
- OPENMP ARCHITECTURE REVIEW BOARD. **OpenMP Application Program Interface**. Disponível em: <https://www.openmp.org>.